

SISTEM MANAJEMEN PASIEN KLINIK

Laporan ini disusun untuk memenuhi tugas UAS

Mata Kuliah Algoritma dan Struktur Data

Dosen Pengampu:

Taufik Ridwan, M.Kom.



Disusun Oleh:

Marsya Bella Puspita Rizky	2510631250019
Meyla Sabrina	2510631250022
Beby Naila Putri	2510631250028
Azkiya Aulia Hidayah	2510631250055

UNIVERSITAS SINGAPERBANGSA KARAWANG

FAKULTAS ILMU KOMPUTER

2025/2026

KATA PENGANTAR

Puji dan syukur kami panjatkan atas kehadiran Allah SWT. Karena dengan rahmat dan karunia-Nya penulis dapat menyelesaikan laporan dengan judul "Sistem Manajemen Pasien Klinik" dengan tepat waktu.

Laporan ini dibuat untuk memenuhi tugas mata kuliah Algoritma dan Struktur Data. Penulis berharap semoga laporan ini dapat menambah pengetahuan dan pengalaman bagi pembaca.

Penulis menyadari bahwa laporan ini jauh dari kata sempurna dan masih terdapat kekurangan. Oleh karena itu penulis sangat mengharapkan saran dan kritik dari semua pihak. Penulis berharap laporan ini dapat bermanfaat bagi semua pembaca.

Karawang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan.....	2
BAB II ANALISIS KEBUTUHAN	3
2.1 Fitur dalam Sistem Manajemen Pasien	3
2.1.1 CRUD Operations.....	3
2.1.2 Searching Features	4
2.1.3 Sorting Features.....	5
2.1.4 Additional Features.....	5
2.2 Struktur Data.....	6
BAB III IMPLEMENTASI	7
3.1 Manajemen Penyimpanan dan Persistensi Data.....	7
3.1.1 ArrayList	7
3.1.2 Persistensi Data Berbasis File (CSV).....	7
3.2 Modul Manajemen Data (CRUD)	8
3.2.1 Create.....	8
3.2.2 Update.....	9
3.2.3 Soft Delete	10
3.3 Implementasi Algoritma Pengurutan (Sorting).....	10
3.3.1 Bubble Sort	10
3.3.2 Selection Sort	11
3.4 Implementasi Algoritma Pencarian (Searching)	12
3.4.1 Linear Search	12

3.4.2	Binary Search.....	12
3.5	Pengelolaan Statistik dan Keamanan Kode.....	13
3.5.1	Logika Metode Statistik.....	13
3.5.2	Robustness (Keamanan File I/O).....	13
BAB IV ANALISIS KOMPLEKSITAS		15
4.1	Analisis Kompleksitas Program.....	15
4.2	Analisis Kompleksitas Setiap Fitur.....	16
4.2.1	Analisis Fitur Tambah Pasien (addPatient)	16
4.2.2	Analisis Fitur Tampilkan Semua Data (ShowAll).....	18
4.2.3	Analisis Fitur Edit Pasien (editPatient)	19
4.2.4	Analisis Fitur Hapus Pasien (deletePatient)	20
4.2.5	Analisis Fitur Cari Berdasarkan Nama (searchByName)	21
4.2.6	Analisis Fitur Cari Berdasarkan ID (SearchByID).....	22
4.2.7	Analisis Fitur Cari Berdasarkan Kategori (searchByCategory)	24
4.2.8	Analisis Fitur Cari Berdasarkan Status (SearchByActive)	25
4.2.9	Analisis Fitur Pengurutan Berdasarkan ID (sortByID)	26
4.2.10	Analisis Fitur Pengurutan Berdasarkan Nama (sortByName)	28
4.2.11	Analisis Fitur Pengurutan Berdasarkan Kunjungan (sortByvisitCount)	29
4.2.12	Analisis Fitur Statistik (statistics).....	30
4.2.13	Analisis Fitur Save CSV (saveToCSV)	32
4.3	Tabel Analisis Kompleksitas Algoritma.....	33
4.4	Kesimpulan Analisis Kompleksitas	34
BAB V PENGUJIAN		36
5.1	Pengujian Menu Ke-1 (CRUD).....	36
5.2	Pengujian Menu Ke-2 (CRUD).....	37
5.3	Pengujian Menu Ke-3 (CRUD).....	38
5.4	Pengujian Menu Ke-4 (CRUD).....	39

5.5	Pengujian Menu Ke-5 (Searching)	39
5.6	Pengujian Menu Ke-6 (Searching)	40
5.7	Pengujian Menu Ke-7 (Searching)	40
5.8	Pengujian Menu Ke-8 (Searching)	41
5.9	Pengujian Menu Ke-9 (Sorting)	42
5.10	Pengujian Menu Ke-10 (Sorting)	43
5.11	Pengujian Menu Ke-11 (Sorting)	44
5.12	Pengujian Menu Ke-12 (Sorting)	44
BAB VI KESIMPULAN		45
6.1	Kesimpulan.....	45

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi informasi saat ini mendorong berbagai sektor untuk beralih dari sistem manual ke sistem digital, termasuk dalam bidang pelayanan kesehatan. Klinik sebagai fasilitas kesehatan tingkat pertama memiliki aktivitas administrasi yang cukup kompleks, seperti pencatatan data pasien, pengelolaan riwayat kunjungan, serta pencarian informasi pasien secara cepat dan akurat.

Namun, dalam prakteknya masih banyak klinik yang mengelola data pasien secara sederhana, bahkan masih bergantung pada pencatatan manual atau sistem yang belum terstruktur dengan baik. Hal ini sering menimbulkan berbagai permasalahan, seperti kesulitan dalam mencari data pasien, proses pengolahan data yang memakan waktu lama, serta potensi terjadinya kesalahan dalam pencatatan.

Selain itu, keterbatasan dalam pengolahan data juga membuat proses analisis, seperti mengetahui jumlah pasien atau status data tertentu, menjadi kurang efektif. Padahal, informasi tersebut cukup penting untuk mendukung pengambilan keputusan dalam pelayanan klinik.

Berdasarkan permasalahan tersebut, diperlukan sebuah sistem yang tidak hanya mampu menyimpan data, tetapi juga dapat mengelola, mencari, dan mengurutkan data secara efisien. Oleh karena itu, pada proyek ini dikembangkan sebuah sistem manajemen pasien klinik yang dilengkapi dengan fitur CRUD (Create, Read, Update, Delete), pencarian data menggunakan metode linear search dan binary search, serta pengurutan data menggunakan algoritma bubble sort dan selection sort.

Dengan adanya sistem ini, diharapkan proses pengelolaan data pasien menjadi lebih terstruktur, cepat, dan mudah digunakan, serta dapat meningkatkan efisiensi kerja dalam lingkungan klinik.

1.2 Tujuan

Adapun tujuan dari pembuatan sistem manajemen pasien klinik ini adalah sebagai berikut:

1. Mengembangkan sistem yang mampu melakukan pengelolaan data pasien secara lengkap melalui fitur CRUD.
2. Mempermudah proses pencarian data pasien berdasarkan nama, ID, maupun kategori dengan metode yang sesuai.
3. Mengimplementasikan algoritma pencarian (searching) dan pengurutan (sorting) dalam sistem untuk meningkatkan efisiensi pengolahan data.
4. Membantu pengguna dalam mengurutkan data pasien berdasarkan kriteria tertentu seperti ID, nama, dan jumlah data.
5. Menyediakan informasi tambahan berupa statistik data untuk mendukung pengambilan keputusan.
6. Mengurangi kesalahan dalam pengelolaan data yang sering terjadi pada sistem manual.

BAB II

ANALISIS KEBUTUHAN

2.1 Fitur dalam Sistem Manajemen Pasien

Adapun beberapa fitur yang diperlukan dalam pembuatan sistem manajemen pasien seperti *CRUD Operations*, *Searching Features*, *Sorting Features*, dan *Additional Features*.

2.1.1 CRUD Operations

CRUD operations adalah singkatan dari *Create*, *Read*, *Update*, dan *Delete*. *CRUD operations* adalah fitur atau operasi dasar dalam pengolahan data di sistem manajemen. *CRUD operations* memudahkan kita dalam pengembangan dan pengolahan data pada aplikasi.

1) *Create*

Fitur *create* dalam *CRUD* adalah untuk menambahkan suatu data baru ke dalam *database*. Dalam sistem manajemen pasien fitur *create* yang diperlukan adalah dapat menambahkan data pasien seperti ID, nama, kategori (poli), status (Aktif/Non-aktif), dan jumlah kunjungan ke dalam file.

2) *Read*

Fitur *read* dalam *CRUD* adalah untuk menampilkan data dalam *class* yang tersimpan dalam *database*. Dalam sistem manajemen pasien fitur *read* yang diperlukan adalah dapat menampilkan semua data pasien yang sudah tercatat atau tersimpan di *database*.

3) *Update*

Fitur *update* dalam *CRUD* adalah fitur untuk memanipulasi atau memperbarui data yang sudah tersimpan dalam *database*. Dalam sistem manajemen pasien fitur *update* yang diperlukan

adalah dapat memperbarui nama, kategori (poli), status (Aktif/Non-aktif), dan jumlah kunjungan berdasarkan ID pasien.

4) *Delete*

Fitur *delete* merupakan fitur untuk menghapus *database* maupun data yang berada di *database*. Dalam sistem manajemen pasien fitur *delete* yang diperlukan adalah fitur *soft delete* yaitu menonaktifkan status pasien berdasarkan ID pasien.

2.1.2 Searching Features

Searching features atau fitur pencarian adalah proses menemukan data atau lokasi (indeks) suatu nilai (*key*) dalam kumpulan data (*array*, *arraylist*, dan lain-lain). Fitur ini memiliki dua jenis, yaitu *Linear search* dan *Binary search*.

Linear search adalah algoritma pencarian sederhana dengan membandingkan elemen satu per-satu dan sering digunakan pada data yang belum berurutan dan berukuran kecil. Sedangkan *Binary search* adalah algoritma pencarian efisien pada data yang sudah berurutan.

Dalam sistem manajemen pasien, fitur pencarian yang diperlukan yaitu:

- 1) Cari berdasarkan Nama: Mencari data pasien berdasarkan nama menggunakan *Linear search*.
- 2) Cari berdasarkan ID: Mencari data pasien berdasarkan ID menggunakan *Binary search*.
- 3) Cari berdasarkan kategori: Mencari data pasien berdasarkan kategori (poli dalam klinik) menggunakan *Linear search*.
- 4) Cari berdasarkan status: Mencari data pasien berdasarkan status (Aktif/Non-aktif) menggunakan *Linear search*.

2.1.3 Sorting Features

Sorting features atau fitur pengurutan adalah proses menyusun elemen data dalam urutan tertentu (menaik atau menurun). Fitur pengurutan merupakan operasi fundamental yang dapat mempermudah pencarian (*Searching*) dan penggabungan (*Merging*).

Sistem manajemen pasien menggunakan dua jenis pengurutan yaitu *Bubble sort* dan *Selection sort*. *Bubble sort* adalah algoritma pengurutan sederhana yang cara kerjanya membandingkan pasangan elemen yang berdekatan dan menukarnya jika urutannya salah. Sedangkan *Selection sort* adalah algoritma pengurutan sederhana dengan membagi array menjadi *sorted* dan *unsorted*, lalu mencari elemen minimum dari bagian *unsorted* dan menempatkannya di akhir bagian *sorted*.

Dalam sistem manajemen pasien, fitur pengurutan yang diperlukan yaitu:

- 1) Pengurutan ID pasien menggunakan *Bubble Sort* secara ascending: Fitur ini mampu mengurutkan ID pasien secara ascending atau dari ID terkecil ke terbesar.
- 2) Pengurutan nama pasien menggunakan *Selection Sort* secara alphabetical: Fitur ini mampu mengurutkan nama pasien secara ascending atau alphabetical.
- 3) Pengurutan kunjungan pasien menggunakan *Bubble Sort* secara descending: Fitur ini mampu mengurutkan urutan kunjungan pasien secara descending atau dari kunjungan terbanyak ke kunjungan tersedikit.

2.1.4 Additional Features

Additional features atau fitur tambahan dalam sistem manajemen pasien ini berupa fitur statistik data. Fitur ini mampu

menampilkan total data pasien yang tersimpan dan total data pasien yang aktif dalam database.

2.2 Struktur Data

Struktur data yang digunakan dalam sistem manajemen pasien ini adalah Arraylist objek. Arraylist merupakan bagian dari Java Collections Framework yang dapat menyimpan elemen secara berurutan dan ukurannya bisa bertambah secara dinamis. Class patient merupakan objek yang menyimpan variabel dengan tipe data yang berbeda-beda (int id, String name, boolean active, dst). Secara singkat Arraylist objek merupakan struktur data utama yang di mana setiap elemen mewakili satu pasien yang memiliki atribut lengkap (ID, Nama, Kategori, Status, dan Jumlah Kunjungan).

Data juga disimpan dalam CSV. CSV (Comma-Separated Values) adalah sebuah format file teks biasa yang digunakan untuk menyimpan data dalam bentuk tabel, di mana setiap nilai atau kolomnya dipisahkan oleh tanda koma (,).

Penggunaan Arraylist objek dipilih karena kita tidak perlu menentukan berapa jumlah maksimal pasien sehingga memudahkan proses penambahan data pasien yang terus bertambah, lalu mudah untuk perulangan (for-each) untuk fitur pencarian, dan mudah untuk memodifikasi suatu data.

BAB III

IMPLEMENTASI

3.1 Manajemen Penyimpanan dan Persistensi Data

Sistem ini menggunakan dua mekanisme penyimpanan untuk memastikan data dapat diakses dengan cepat di memori, namun tetap aman di penyimpanan permanen.

3.1.1 ArrayList

```
static ArrayList<Patient> patients = new ArrayList<>();
```

Menggunakan *ArrayList<Patient>* sebagai struktur data utama selama program berjalan. Pemilihan *ArrayList* didasarkan pada sifatnya yang *resizeable*, sehingga alokasi memori akan bertambah secara otomatis seiring bertambahnya data pasien tanpa risiko array *overflow*.

3.1.2 Persistensi Data Berbasis File (CSV)

```
static void saveToCSV() {  
    try {  
        FileWriter writer = new FileWriter(fileName: "data.csv");  
        writer.write(str: "ID>Nama,Kategori>Status,Kunjungan\n");  
        for (Patient p : patients) {  
            writer.write(  
                p.id + "," +  
                p.name + "," +  
                p.category + "," +  
                p.active + "," +  
                p.visitCount + "\n"  
            );  
        }  
        writer.close();  
        System.out.println(x: "Data berhasil disimpan ke CSV!");  
    } catch (IOException e) {  
        System.out.println(e);  
    }  
}
```

Program ini menggunakan *FileWriter* dengan mode *append*. Setiap kali data pasien baru ditambahkan, sistem secara otomatis mencatat ke dalam file *data.csv*. Hal ini memastikan bahwa data tidak hilang (persisten) meskipun aplikasi ditutup. Teknik ini jauh lebih aman karena mencegah terjadinya penumpukkan data ganda (redudansi) di dalam file CSV setiap kali program melakukan sinkronisasi.

3.2 Modul Manajemen Data (CRUD)

Modul ini menangani interaksi pengguna dengan database pasien.

3.2.1 Create

Operasi ini tidak hanya menyimpan data ke dalam List, tetapi juga melakukan sinkronisasi dengan database eksternal berupa file CSV. Proses penambahan data pada metode *addPatient()* dilengkapi dengan dua lapis validasi untuk menjaga integritas data (menghindari duplikasi ID dan Nama).

```
static void addPatient(){
    System.out.print("ID: ");
    int id = input.nextInt();
    input.nextLine();

    boolean idSudahAda = false;
    for(Patient p : patients){
        if(p.id == id){
            idSudahAda = true;
            break;
        }
    }
    if(idSudahAda){
        System.out.println("ID: " + id + " sudah terdaftar! Gagal menambahkan pasien.");
        return;
    }

    System.out.print("Nama: ");
    String name = input.nextLine();

    boolean namaSudahAda = false;
    for(Patient p : patients){
        if(p.name.equalsIgnoreCase(name)){
            namaSudahAda = true;
            break;
        }
    }
    if(namaSudahAda){
        System.out.println("Nama: " + name + " sudah terdaftar! Gagal menambahkan pasien.");
        return;
    }
}
```

```

System.out.print(s: "Kategori [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: ");
String category = input.nextLine();

System.out.print(s: "Status [1 = Aktif, 0 = Nonaktif]: ");
Boolean active = input.nextInt() == 1;

System.out.print(s: "Jumlah kunjungan: ");
int visitCount = input.nextInt();
patients.add(new Patient(id, name, category, active, visitCount));
saveToCSV();
System.out.println(x: "Data pasien berhasil ditambahkan!");
}

```

- 1) *input.nextLine()* digunakan tepat setelah *nextInt()* untuk membuang karakter sisa newline (`\n`) dari memori buffer agar tidak memicu kegagalan input teks (input skipping) pada form Nama.
- 2) Logika *input.nextLine() == 1* mengevaluasi input integer secara langsung menjadi nilai boolean (*true* atau *false*) untuk menyederhanakan interaksi pengguna.
- 3) Fungsi *saveToCSV()* dipanggil di akhir proses untuk menjamin data di memori utama dan media penyimpanan selalu sinkron (Immediate Persistence).

3.2.2 Update

Proses pembaruan data bekerja mencari objek target berdasarkan ID menggunakan sequential scan, kemudian memperbarui nilainya melalui konsep Pass by Reference.

```

static void editPatient(){
    System.out.print(s: "Masukkan ID: ");
    int id = input.nextInt();

    for(Patient p : patients){
        if(p.id == id){
            input.nextLine();
            System.out.print(s: "Nama: ");
            p.name = input.nextLine();
            System.out.print(s: "Kategori baru [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: ");
            p.category = input.nextLine();
            System.out.print(s: "Status baru [1 = Aktif, 0 = Nonaktif]: ");
            p.active = Boolean.parseBoolean(input.nextLine());
            System.out.print(s: "Jumlah kunjungan baru: ");
            p.visitCount = Integer.parseInt(input.nextLine());

            break;
        }
    }
}

```

Di dalam perulangan, variabel *p* menyimpan alamat memori objek asli di dalam *ArrayList*. Oleh karena itu, modifikasi langsung pada properti objek seperti *p.name = input.nextLine()* akan langsung mengubah data utama tanpa perlu memanggil metode penggantian elemen secara manual.

3.2.3 Soft Delete

Proses penghapusan tidak membuang objek secara fisik dari list, melainkan hanya menonaktifkan status operasional pasien.

```
static void deletePatient() {  
    System.out.print(s, "ID: ");  
    int id = input.nextInt();  
  
    for (Patient p : patients) {  
        if (p.id == id) {  
            p.active = false;  
        }  
    }  
    saveToCSV();  
}
```

Sistem konsep Soft Delete sangat penting dalam sistem rekam medis untuk menjaga Audit Trail (jejak audit). Data pasien yang tidak aktif tetap tersimpan di dalam berkas CSV untuk pelaporan historis rumah sakit tanpa merusak struktur indeks memori.

3.3 Implementasi Algoritma Pengurutan (Sorting)

Sistem menerapkan dua algoritma pengurutan klasik yang dimodifikasi untuk memproses tipe data objek yang berbeda. Proses pertukaran (swapping) menggunakan variabel bantu.

3.3.1 Bubble Sort

Bubble Sort bekerja dengan membandingkan dua elemen yang bersebelahan secara berulang dan menukarnya jika kondisinya terpenuhi.

1) Ascending

```
static void sortByID() {
    for (int i = 0; i < patients.size() - 1; i++) {
        for (int j = 0; j < patients.size() - i - 1; j++) {
            if (patients.get(j).id > patients.get(j + 1).id) {
                Patient temp = patients.get(j);
                patients.set(j, patients.get(j + 1));
                patients.set(j + 1, temp);
            }
        }
    }
}
```

Operator `>` digunakan untuk membandingkan atribut integer *id*. Jika elemen kiri lebih besar dari elemen kanan, posisi objek akan ditukar menggunakan objek bantuan *Patient temp*. Pengurutan secara menaik (ascending) ini wajib dijalankan sebelum memanggil fungsi Binary Search.

2) Descending

```
static void sortByvisitCount() {
    for (int i = 0; i < patients.size() - 1; i++) {
        for (int j = 0; j < patients.size() - i - 1; j++) {
            if (patients.get(j).visitCount < patients.get(j + 1).visitCount) {
                Patient temp = patients.get(j);
                patients.set(j, patients.get(j + 1));
                patients.set(j + 1, temp);
            }
        }
    }
}
```

Menggunakan logika pembandingan `<` agar objek dengan nilai *visitCount* yang lebih kecil tenggelam ke belakang, sehingga menghasilkan daftar urutan pasien paling loyal (kunjungan terbanyak) di baris terdepan.

3.3.2 Selection Sort

Selection Sort bekerja dengan mencari elemen dengan nilai terkecil di seluruh sisa list pada setiap putaran, lalu menukarnya ke posisi target.

```
static void sortByName() {
    for (int i = 0; i < patients.size() - 1; i++) {
        int min = i;
        for (int j = i + 1; j < patients.size(); j++) {
            if (patients.get(j).name.compareTo(patients.get(min).name) < 0) {
                min = j;
            }
        }
        Patient temp = patients.get(i);
        patients.set(i, patients.get(min));
        patients.set(min, temp);
    }
}
```


Karena nama bertipe data *String*, operasi perbandingan harus menggunakan metode bawaan *.compareTo()*. Jika hasil kembalian bernilai <0 , berarti teks pada *patient.get(j)* secara urutan alfabetis berada di depan target pembandingan. Keunggulan Selection Sort di sini adalah efisiensi jumlah penukaran objek (swap count hanya senilai $O(n)$) yang membuatnya lebih hemat resource memori dibanding Bubble Sort untuk manipulasi data berbasis teks.

3.4 Implementasi Algoritma Pencarian (Searching)

Sistem mengimplementasikan dua metode pencarian yang disesuaikan dengan keunikan data yang dicari.

3.4.1 Linear Search

Pencarian sekuensial memindai elemen dari indeks awal sampai akhir satu per satu. Metode ini digunakan pada fitur *searchByName()*, *searchByCategory()*, dan *searchByActive()*.

```
for (Patient p : patients) {  
    if (p.category.equalsIgnoreCase(category)) {  
        p.display();  
        ditemukan = true;  
    }  
}
```

Linear Search sangat efektif digunakan untuk mencari atribut data yang tidak unik (bisa memiliki banyak hasil yang sama, seperti kategori “Umum” atau status “Aktif”). Metode *equalsIgnoreCase()* diterapkan agar pencarian bersifat case-insensitive (tidak sensitif huruf kapital) demi kenyamanan pengguna.

3.4.2 Binary Search

Algoritma pencarian efisien tinggi yang bekerja dengan membagi area pencarian menjadi dua bagian secara terus-menerus.

```

static void searchByID() {
    sortByID(); // syarat binary search

    System.out.print("ID: ");
    int target = input.nextInt();

    int left = 0, right = patients.size() - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (patients.get(mid).id == target) {
            patients.get(mid).display();
            return;
        } else if (patients.get(mid).id < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }
    System.out.println("ID Pasien " + target + " Tidak ditemukan");
}

```

Kompleksitas waktu algoritma ini adalah $O(\log n)$, jauh lebih cepat daripada Linear Search $O(n)$ saat menangani ribuan data. Di dalam kode ini, fungsi *sortByID()* dipanggil secara otomatis di awal metode sebagai jaminan mutlak bahwa data sudah terurut secara ascending sebelum proses pembagian indeks (*mid*) dieksekusi.

3.5 Pengelolaan Statistik dan Keamanan Kode

Bagian ini menerangkan penanganan kalkulasi ringkasan data serta proteksi error penulisan file.

3.5.1 Logika Metode Statistik

```

static void statistics() {
    int total = patients.size();
    int active = 0;

    for (Patient p : patients) {
        if (p.active) active++;
    }

    System.out.println("Total: " + total);
    System.out.println("Aktif: " + active);
}

```

Fungsi ini memanfaatkan property size dari *ArrayList* untuk mengetahui total populasi data pasien keseluruhan dan menggunakan iterasi kondisional untuk menyaring jumlah pasien yang benar-benar masih berstatus aktif saja secara akurat.

3.5.2 Robustness (Keamanan File I/O)

Seluruh penulisan berkas pada *saveToCSV()* diproteksi menggunakan blok *try-catch* dengan menangkap objek kesalahan

IOException. Hal ini memastikan bahwa apabila terjadi galat sistem (misal: file terkunci oleh sistem operasi atau memori penyimpanan penuh), program tidak akan berhenti mendadak (crash), melainkan menampilkan log pesan kesalahan secara aman di terminal.

BAB IV

ANALISIS KOMPLEKSITAS

4.1 Analisis Kompleksitas Program

Analisis kompleksitas pada Sistem Manajemen Pasien Klinik dilakukan untuk mengetahui tingkat efisiensi algoritma yang digunakan dalam pengolahan data pasien. Analisis ini bertujuan untuk mengukur pengaruh pertambahan jumlah data terhadap waktu eksekusi program. Semakin kecil nilai kompleksitas suatu algoritma, maka semakin cepat dan efisien program dalam memproses data.

Pengukuran kompleksitas dalam analisis ini menggunakan notasi Big-O (Big-O Notation). Notasi Big-O digunakan untuk menggambarkan pertumbuhan waktu eksekusi suatu algoritma berdasarkan jumlah data (n) yang diproses. Dalam notasi ini, hanya pertumbuhan operasi yang paling dominan yang diperhatikan, sedangkan konstanta dan operasi kecil lainnya diabaikan karena tidak terlalu berpengaruh pada data berukuran besar.

Analisis performa ini ditinjau berdasarkan dua kondisi utama, yaitu *Best Case* (kondisi ideal dengan jumlah operasi minimum) dan *Worst Case* (kondisi terburuk dengan jumlah operasi maksimal).

Sistem Manajemen Pasien Klinik ini menggunakan beberapa algoritma dasar, seperti algoritma pencarian (searching) dan pengurutan (sorting). Algoritma pencarian yang digunakan meliputi Linear Search dan Binary Search, sedangkan algoritma pengurutan menggunakan Bubble Sort dan Selection Sort. Setiap algoritma memiliki tingkat kompleksitas yang berbeda tergantung pada proses dan jumlah data yang diolah.

Berikut merupakan tabel perbandingan kompleksitas waktu dari algoritma yang digunakan pada implementasi Sistem Manajemen Pasien Klinik.

Algoritma	Best Case	Average Case	Worst Case	Space (Worst)
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$ iteratif
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$

Berdasarkan tabel tersebut, dapat diketahui bahwa algoritma pencarian dan pengurutan memiliki tingkat efisiensi yang berbeda-beda. Algoritma dengan kompleksitas lebih kecil, seperti *Binary Search*, cenderung memiliki performa lebih baik dibandingkan algoritma dengan kompleksitas kuadrat seperti *Bubble Sort* dan *Selection Sort* ketika jumlah data semakin besar. Oleh karena itu, pemilihan algoritma yang tepat sangat berpengaruh terhadap kinerja Sistem Manajemen Pasien Klinik agar tetap optimal saat memproses data dalam jumlah banyak.

4.2 Analisis Kompleksitas Setiap Fitur

4.2.1 Analisis Fitur Tambah Pasien (addPatient)

Fitur ini diimplementasikan melalui fungsi `addPatient` yang berfungsi untuk menambahkan data pasien baru ke dalam sistem. Data yang dimasukkan meliputi ID pasien, nama, kategori, status pasien, dan jumlah kunjungan pasien. Sebelum proses penyimpanan dilakukan, sistem melakukan pengecekan ID dan nama pasien untuk menghindari terjadinya duplikasi data.

Proses pencarian data menggunakan algoritma Linear Search karena proses pencarian dilakukan dengan memeriksa data pasien satu per satu secara berurutan pada struktur data `ArrayList`. Algoritma Linear Search dipilih karena memiliki implementasi yang sederhana dan sesuai untuk struktur data `ArrayList` yang belum terurut.

Potongan kode program:

```
//CRUD
static void addPatient(){
    System.out.print("ID: ");
    int id = input.nextInt();
    input.nextLine();

    boolean idSudahAda = false;
    for(Patient p : patients){
        if(p.id == id){
            idSudahAda = true;
            break;
        }
    }
    if(idSudahAda){
        System.out.println("ID: " + id + " sudah terdaftar! Gagal menambahkan pasien.");
        return;
    }

    System.out.print("Nama: ");
    String name = input.nextLine();

    boolean namaSudahAda = false;
    for(Patient p : patients){
        if(p.name.equalsIgnoreCase(name)){
            namaSudahAda = true;
            break;
        }
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi apabila ID atau nama pasien yang dimasukkan tidak ditemukan di dalam sistem (merupakan data yang benar-benar baru). Kondisi ini memaksa program untuk melakukan iterasi dan memeriksa seluruh elemen di dalam ArrayList sebanyak n elemen untuk kedua proses pengecekan.

Perhitungan kompleksitas *worst case* dijabarkan sebagai berikut:

$$T(n) = 1 + 1 + n + n + 1 + n + n + 1 + 1 + 1$$

$$T(n) = 4n + 7$$

Berdasarkan notasi Big-O, konstanta dan koefisien diabaikan, sehingga kompleksitasnya adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi ketika data pasien langsung dapat ditambahkan tanpa perlu melakukan iterasi pencarian yang panjang, sehingga proses berjalan dalam operasi konstan.

$$T(n) = 1 + 1 + 1$$

$$T(n) = 3$$

Maka kompleksitas *best case* adalah $O(1)$.

4.2.2 Analisis Fitur Tampilkan Semua Data (ShowAll)

Metode ShowAll digunakan untuk menampilkan seluruh data pasien yang tersimpan dalam sistem. Fitur ini menggunakan algoritma Traversal karena program harus membaca seluruh elemen data pasien satu persatu. Algoritma traversal dipilih karena sistem harus membaca seluruh elemen data tanpa proses pencarian maupun pengurutan.

Potongan kode program:

```
static void showAll(){
    for(Patient p : patients){
        p.display();
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika sistem harus menampilkan seluruh data pasien yang tersimpan sebanyak n elemen.

Perhitungan kompleksitas:

$$T(n) = 1 + n + n$$

$$T(n) = 2n + 1$$

Maka, kompleksitas *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Pada algoritma traversal untuk menampilkan data, sistem akan selalu membaca seluruh elemen dari awal hingga akhir tanpa memperdulikan kondisi data. Oleh karena itu, jumlah operasi yang dilakukan pada *best case* tetap sama dengan *worst case*.

$$T(n) = 1 + n + n$$

$$T(n) = 2n + 1$$

Maka, kompleksitas pada *best case* juga adalah $O(n)$.

4.2.3 Analisis Fitur Edit Pasien (*editPatient*)

Metode *editPatient* digunakan untuk memperbarui data pasien berdasarkan ID pasien. Sebelum data dapat diubah, sistem melakukan pencarian ID terlebih dahulu sebelum data pasien dapat diperbarui. Fitur ini menggunakan algoritma Linear Search karena proses pencarian dilakukan secara berurutan pada seluruh elemen *ArrayList*. Algoritma ini sesuai digunakan karena struktur data yang digunakan belum memiliki indeks khusus untuk pencarian data.

Potongan kode program:

```
static void editPatient(){
    System.out.print("Masukkan ID: ");
    int id = input.nextInt();

    for(Patient p : patients){
        if(p.id == id){
            input.nextLine();
            System.out.print("Nama: ");
            p.name = input.nextLine();
            System.out.print("Kategori baru [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: ");
            p.category = input.nextLine();
            System.out.print("Status baru [1 = Aktif, 0 = Nonaktif]: ");
            p.active = Boolean.parseBoolean(input.nextLine());
            System.out.print("Jumlah kunjungan baru: ");
            p.visitCount = Integer.parseInt(input.nextLine());
            break;
        }
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika ID pasien yang dicari berada pada elemen terakhir atau tidak ditemukan sehingga sistem harus memeriksa seluruh elemen *ArrayList*.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1 + 1$$

$$T(n) = 2n + 5$$

Maka kompleksitas *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi ketika data duplikat langsung ditemukan pada elemen pertama sehingga proses validasi dapat dihentikan lebih cepat.

Perhitungan kompleksitas :

$$T(n) = 1 + 1 + 1 + 1 + 1 + 1 + 1$$

$$T(n) = 7$$

Maka kompleksitas pada best case adalah $O(1)$.

4.2.4 Analisis Fitur Hapus Pasien (deletePatient)

Metode deletePatient digunakan untuk menghapus data pasien secara logis (soft delete) dengan cara mengubah status aktivitas pasien (p.active) menjadi nonaktif (false) berdasarkan ID. Fitur ini menggunakan algoritma Linear Search untuk mencari ID pasien yang akan dinonaktifkan.

Potongan kode program:

```
static void deletePatient() {  
    System.out.print("ID: ");  
    int id = input.nextInt();  
  
    for (Patient p : patients) {  
        if (p.id == id) {  
            p.active = false;  
        }  
    }  
    saveToCSV();  
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika data pasien yang akan dinonaktifkan berada pada posisi terakhir atau tidak ditemukan sehingga seluruh elemen harus diperiksa.

Perhitungan kompleksitas:

$$T(n) = 1 + n + n + 1 + 1$$

$$T(n) = 2n + 3$$

Maka, kompleksitas pada *worst case* juga adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi ketika data langsung ditemukan pada indeks pertama sehingga proses pencarian dapat dihentikan lebih cepat.

Perhitungan kompleksitas :

$$T(n) = 1 + 1 + 1 + 1 + 1$$

$$T(n) = 5$$

Maka kompleksitas *best case* adalah $O(1)$.

4.2.5 Analisis Fitur Cari Berdasarkan Nama (*searchByName*)

Fitur ini menggunakan algoritma Linear Search untuk mencocokkan input nama secara sekuensial terhadap variabel name dalam ArrayList. Struktur perulangan for-each pada kode ini sengaja dirancang tanpa instruksi break agar sistem dapat menyisir seluruh isi data dan menampilkan semua pasien yang kemungkinan memiliki kesamaan nama (redundancy).

Potongan kode program:

```
// SEARCH
static void searchByName() {
    System.out.print("Nama: ");
    String name = input.nextLine();

    boolean ditemukan = false;

    for (Patient p : patients) {
        if (p.name.equalsIgnoreCase(name)) {
            p.display();
            ditemukan = true;
        }
    }
    if(!ditemukan) {
        System.out.println("Nama Pasien " + name + " Tidak Ditemukan.");
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika sistem harus menampilkan seluruh data pasien yang tersimpan sebanyak n elemen.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka, kompleksitas pada *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi apabila nama pasien yang dicari langsung ditemukan pada indeks pertama perulangan. Namun, karena tidak adanya instruksi pemutus seperti *break* demi memastikan seluruh pemilik nama yang sama terdeteksi, program tetap terpaksa mengiterasi sisa elemen data yang ada hingga selesai.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka, kompleksitas pada *best case* adalah $O(n)$.

4.2.6 Analisis Fitur Cari Berdasarkan ID (SearchByID)

Fitur *searchByID* digunakan untuk mencari pasien berdasarkan ID pasien. Pada fitur ini digunakan kombinasi algoritma Bubble Sort dan Binary Search. Bubble Sort digunakan untuk memastikan data terurut berdasarkan ID, sedangkan Binary Search mempercepat proses pencarian setelah data terurut. Alasan penggunaan kombinasi ini adalah karena Binary Search hanya dapat digunakan pada data yang sudah terurut.

Potongan kode program:

```

static void searchByID() {
    sortByID(); // syarat binary search

    System.out.print("ID: ");
    int target = input.nextInt();

    int left = 0, right = patients.size() - 1;

    while (left <= right) {
        int mid = (left + right) / 2;

        if (patients.get(mid).id == target) {
            patients.get(mid).display();
            return;
        } else if (patients.get(mid).id < target) {
            left = mid + 1;
        } else {
            right = mid - 1;
        }
    }

    System.out.println("ID Pasien " + target + " Tidak ditemukan");
}

```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi apabila data pasien berada dalam kondisi acak terbalik (descending) saat proses pengurutan, dan target ID yang dicari berada di ujung akhir pembelahan atau tidak ditemukan oleh Binary Search. Pada kondisi ini, Bubble Sort akan mengambil performa terburuknya yaitu $O(n^2)$ dan Binary Search akan mengambil performa $O(\log n)$.

Perhitungan kompleksitas:

$$T(n) = T_{Bubble} + T_{Binary}$$

$$T(n) = n^2 + \log n + 1 + 1 + 1$$

$$T(n) = n^2 + \log n + 3$$

Berdasarkan notasi Big-O, hanya suku dominan dengan pertumbuhan terbesar yang diambil, yaitu n^2 . Dengan demikian, kompleksitas waktu worst case disederhanakan menjadi $O(n^2)$ (Kuadratik).

2. Kasus Terbaik (*Best Case*)

Best case tercapai apabila data pasien di dalam penyimpanan ternyata sudah dalam keadaan terurut sebelum fungsi dipanggil, dan target ID yang dicari kebetulan langsung ditemukan tepat di titik tengah pada pembelahan pertama Binary Search. Pada

skenario ini, Bubble Sort hanya melakukan satu kali operasional pemindaian konfirmasi sebesar $O(n)$, sedangkan Binary Search langsung selesai dalam $O(1)$ operasi berkat adanya instruksi return.

Perhitungan kompleksitas:

$$T(n) = T_{Bubble} + T_{Binary}$$

$$T(n) = n + 1 + 1 + 1 + 1$$

$$T(n) = n + 4$$

Melalui penyederhanaan notasi Big-O, suku konstan diabaikan dan suku pangkat tertinggi yang diambil, sehingga nilai kompleksitas waktu pada best case total adalah $O(n)$.

4.2.7 Analisis Fitur Cari Berdasarkan Kategori (searchByCategory)

Metode searchByCategory menggunakan algoritma Linear Search untuk mencocokkan input kategori secara sekuensial terhadap variabel category di dalam ArrayList. Mirip dengan fitur pencarian nama, perulangan for-each pada kode ini tidak menggunakan instruksi break agar sistem dapat mengumpulkan dan menampilkan seluruh pasien yang berada dalam kelompok kategori yang sama.

Potongan kode program:

```
static void searchByCategory() {
    System.out.print("Masukkan Kategori yang Dicari [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: ");
    String category = input.nextLine();

    boolean ditemukan = false;

    for (Patient p : patients) {
        if (p.category.equalsIgnoreCase(category)) {
            p.display();
            ditemukan = true;
        }
    }
    if(!ditemukan) {
        System.out.println("Pasien dengan Kategori " + category + " Tidak Ditemukan.");
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika sistem harus menampilkan seluruh data pasien yang tersimpan sebanyak n elemen.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka, kompleksitas pada *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case secara teoritis terjadi apabila kategori pasien yang dicari langsung ditemukan pada indeks pertama perulangan. Namun, karena tidak adanya instruksi pemutus seperti break demi memastikan seluruh pasien dengan kategori sejenis teridentifikasi, program tetap wajib mengiterasi sisa elemen data yang ada hingga selesai.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka, kompleksitas pada *best case* juga adalah $O(n)$.

4.2.8 Analisis Fitur Cari Berdasarkan Status (SearchByActive)

Fitur searchByActive menggunakan algoritma Linear Search untuk mencocokkan input status keaktifan secara sekuensial terhadap variabel boolean active di dalam ArrayList. Sama halnya dengan pencarian nama dan kategori, perulangan for-each pada kode ini dirancang tanpa instruksi break agar sistem dapat menyisir seluruh elemen dan menampilkan semua pasien yang memiliki status keaktifan yang sama (aktif atau nonaktif).

Potongan kode program:

```

static void searchByActive(){
    System.out.print("Masukkan Status yang Dicari [1 = Aktif, 0 = Nonaktif]: ");
    boolean active = input.nextInt() == 1;

    boolean ditemukan = false;

    for(Patient p : patients){
        if(p.active == (active)){
            p.display();
            ditemukan = true;
        }
    }
    if(!ditemukan){
        System.out.println("Tidak ada pasien dengan status tersebut.");
    }
}

```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika sistem harus menampilkan seluruh data pasien yang tersimpan sebanyak n elemen.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka kompleksitas pada *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi saat data dengan status keaktifan yang dicari ditemukan pada indeks pertama. Namun, karena tidak ada instruksi break demi mendeteksi seluruh pasien berstatus sama, program tetap harus melakukan iterasi hingga elemen terakhir.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka kompleksitas pada *Best case* juga adalah $O(n)$.

4.2.9 Analisis Fitur Pengurutan Berdasarkan ID (sortByID)

Fitur ini menggunakan algoritma Bubble Sort untuk mengurutkan data pasien berdasarkan ID secara menaik (ascending). Algoritma Bubble Sort dipilih karena memiliki

implementasi yang sederhana dan mudah dipahami, sehingga sesuai digunakan pada pengolahan data berskala kecil hingga menengah. Proses pengurutan dilakukan menggunakan dua lapis perulangan bersarang (nested loop), di mana setiap elemen yang berdekatan dibandingkan secara berulang dan ditukar posisinya apabila urutannya tidak sesuai.

Potongan kode program:

```
// SORTING
static void sortByID() {
    for (int i = 0; i < patients.size() - 1; i++) {
        for (int j = 0; j < patients.size() - i - 1; j++) {
            if (patients.get(j).id > patients.get(j + 1).id) {

                Patient temp = patients.get(j);
                patients.set(j, patients.get(j + 1));
                patients.set(j + 1, temp);
            }
        }
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika data pasien berada dalam urutan yang berlawanan dengan hasil pengurutan yang diinginkan, sehingga setiap elemen harus dibandingkan dan dipindahkan ke posisi yang sesuai. Pada algoritma Bubble Sort, proses ini dilakukan menggunakan dua perulangan bersarang yang menyebabkan jumlah perbandingan meningkat seiring bertambahnya jumlah data.

Perhitungan kompleksitas:

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

Berdasarkan notasi Big-O, kompleksitas waktu pada worst case disederhanakan menjadi $O(n^2)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi apabila data pasien sebenarnya sudah tersusun secara menaik sebelum proses sorting dijalankan. Akan tetapi, implementasi Bubble Sort pada sistem ini belum menggunakan mekanisme penghentian lebih awal (*swap flag*), sehingga seluruh proses perbandingan tetap dijalankan sampai perulangan selesai.

Perhitungan kompleksitas:

$$T(n) = \frac{n(n-1)}{2}$$

Kompleksitas *best Case* disederhanakan menjadi $O(n^2)$.

4.2.10 Analisis Fitur Pengurutan Berdasarkan Nama (*sortByName*)

Fitur ini menggunakan algoritma Selection Sort untuk mengurutkan data pasien berdasarkan nama secara alfabetis (*ascending*). Selection Sort dipilih karena jumlah operasi pertukaran data (*swap*) lebih sedikit dibandingkan Bubble Sort.

Potongan kode program:

```
static void sortByName() {
    for (int i = 0; i < patients.size() - 1; i++) {
        int min = i;
        for (int j = i + 1; j < patients.size(); j++) {
            if (patients.get(j).name.compareTo(patients.get(min).name) < 0) {
                min = j;
            }
        }
        Patient temp = patients.get(i);
        patients.set(i, patients.get(min));
        patients.set(min, temp);
    }
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case pada Selection Sort terjadi baik ketika data acak terbalik (*descending*) maupun dalam kondisi lainnya. Hal ini disebabkan karena algoritma ini selalu memindai seluruh sisa elemen melalui *inner loop* untuk memastikan nilai minimum yang sebenarnya telah ditemukan sebelum melakukan operasi pertukaran (*swap*).

Perhitungan kompleksitas:

$$T(n) = (n-1) + (n-2) + (n-3) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

Dengan demikian, kompleksitas *Worst Case* disederhanakan menjadi $O(n^2)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi ketika data nama pasien sudah tersusun secara alfabetis sebelum proses pengurutan dilakukan. Meskipun demikian, Selection Sort tetap harus menelusuri seluruh elemen yang belum terurut untuk memastikan posisi nilai minimum pada setiap iterasi.

Karena jumlah perbandingan tidak berubah, kompleksitas waktu pada kondisi terbaik tetap $O(n^2)$.

4.2.11 Analisis Fitur Pengurutan Berdasarkan Kunjungan (sortByvisitCount)

Fitur ini menggunakan algoritma Bubble Sort untuk mengurutkan data pasien berdasarkan jumlah kunjungan secara menurun (descending). Bubble Sort dipilih karena implementasinya sederhana dan mudah dipahami untuk data berskala kecil hingga menengah. Logika perulangan bersarang (nested loop) pada fungsi ini membandingkan dua elemen yang berdampingan dan menukarnya jika nilai elemen kiri lebih kecil dari elemen kanan, sehingga pasien dengan jumlah kunjungan tertinggi akan bergeser ke posisi paling awal.

Potongan kode program:

```

static void sortByvisitCount() {
    for (int i = 0; i < patients.size() - 1; i++) {
        for (int j = 0; j < patients.size() - i - 1; j++) {
            if (patients.get(j).visitCount < patients.get(j + 1).visitCount) {
                Patient temp = patients.get(j);
                patients.set(j, patients.get(j + 1));
                patients.set(j + 1, temp);
            }
        }
    }
}

```

1. Kasus Terburuk (*Worst Case*)

Worst case terjadi ketika data pasien berada dalam urutan menaik (ascending), yang merupakan kebalikan dari target hasil akhir. Kondisi ini memaksa seluruh iterasi outer loop dan inner loop berjalan maksimal, serta mengeksekusi operasi pertukaran data (swap) di setiap pengecekan.

Perhitungan kompleksitas:

$$T(n) = (n - 1) + (n - 2) + (n - 3) + \dots + 1$$

$$T(n) = \frac{n(n-1)}{2}$$

Maka kompleksitas *worst Case* disederhanakan menjadi $O(n^2)$.

2. Kasus Terbaik (*Best Case*)

Best case terjadi ketika data kunjungan pasien sudah berada dalam urutan menurun sesuai hasil yang diharapkan. Namun, karena algoritma tetap menjalankan seluruh proses perbandingan tanpa pemeriksaan penghentian awal, jumlah iterasi tidak mengalami perubahan. Oleh sebab itu, kompleksitas pada *Best case* tetap $O(n^2)$.

4.2.12 Analisis Fitur Statistik (statistics)

Fitur statistik mengimplementasikan algoritma Traversal. Algoritma ini bekerja dengan cara mengunjungi, memeriksa, dan memproses setiap elemen data di dalam ArrayList secara

berurutan, mulai dari indeks pertama hingga indeks terakhir tanpa ada yang dilewati. Proses traversal diperlukan karena sistem harus memeriksa seluruh data pasien untuk menghitung jumlah pasien aktif maupun total pasien secara akurat.

Potongan kode program:

```
// STATISTIK
static void statistics() {
    int total = patients.size();
    int active = 0;

    for (Patient p : patients) {
        if (p.active) active++;
    }

    System.out.println("Total: " + total);
    System.out.println("Aktif: " + active);
}
```

1. Kasus Terburuk (*Worst Case*)

Worst case pada fitur ini bernilai sama dengan skenario operasional normalnya, dimana sistem harus melakukan iterasi penuh untuk memeriksa variabel status keaktifan (*p.active*) dari seluruh *n* data pasien yang tersimpan guna mendapatkan hasil perhitungan yang akurat.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka kompleksitas pada *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case pada fungsi statistik ini tetap mengharuskan sistem mengiterasi seluruh elemen dari awal hingga akhir (*n*). Meskipun seluruh data pasien sudah berstatus aktif sejak awal maupun sebaliknya, komputer tidak dapat melewatkan satupun indeks data demi menjamin validitas nilai agregat angka yang ditampilkan.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + n + 1 + 1$$

$$T(n) = 2n + 4$$

Maka kompleksitas pada *best case* juga adalah $O(n)$.

4.2.13 Analisis Fitur Save CSV (saveToCSV)

Metode `saveToCSV` digunakan untuk melakukan persistensi data dengan menuliskan seluruh objek pasien yang berada di dalam `ArrayList` ke dalam berkas eksternal format CSV (`data.csv`). Proses penulisan dilakukan secara sekuensial baris demi baris, di mana setiap atribut pasien dipisahkan menggunakan tanda koma.

```
static void saveToCSV() {
    try {
        FileWriter writer = new FileWriter("data.csv");
        writer.write("ID>Nama,Kategori>Status,Kunjungan\n");
        for (Patient p : patients) {
            writer.write(
                p.id + "," +
                p.name + "," +
                p.category + "," +
                p.active + "," +
                p.visitCount + "\n"
            );
        }
        writer.close();

        System.out.println("Data berhasil disimpan ke CSV!");
    } catch (IOException e) {
        System.out.println(e);
    }
}
```

Potongan kode program:

1. Kasus Terburuk (*Worst Case*)

Worst case pada fungsi ini bernilai sama dengan skenario operasional normalnya. Untuk memastikan seluruh data tersimpan dengan utuh tanpa ada yang terlewat, sistem diwajibkan melakukan iterasi penuh dan melakukan operasi I/O (Input/Output) penulisan berkas terhadap seluruh n entitas pasien yang ada di dalam memori.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + 1 + 1$$

$$T(n) = n + 4$$

Maka kompleksitas pada *worst case* adalah $O(n)$.

2. Kasus Terbaik (*Best Case*)

Best case pada fungsi penulisan data ini tetap berada pada tingkat performa yang sama dengan *worst case*. Komputer tidak memiliki instruksi pemutus ataupun jalan pintas untuk mempercepat penulisan, karena setiap elemen data bernilai krusial untuk disimpan ke dalam media penyimpanan fisik sekunder.

Perhitungan kompleksitas:

$$T(n) = 1 + 1 + n + 1 + 1$$

$$T(n) = n + 4$$

Maka kompleksitas pada *best case* juga adalah $O(n)$.

4.3 Tabel Analisis Kompleksitas Algoritma

No	Fitur	Algoritma	Worst Case	Best Case
1.	Tambah Pasien	ArrayList Add	$O(n)$	$O(1)$
2.	Tampilkan Semua Data	Traversal	$O(n)$	$O(n)$
3.	Edit Pasien	Linear Search	$O(n)$	$O(1)$
4.	Hapus Pasien	Linear Search	$O(n)$	$O(1)$
5.	Cari Berdasarkan Nama	Linear Search	$O(n)$	$O(n)$
6.	Cari Berdasarkan ID	Bubble Sort + Binary Search	$O(n^2)$	$O(n)$

7.	Cari Berdasarkan Kategori	Linear Search	$O(n)$	$O(n)$
8.	Cari Berdasarkan Status	Linear Search	$O(n)$	$O(n)$
9.	Pengurutan Berdasarkan ID	Bubble Sort	$O(n^2)$	$O(n^2)$
10.	Pengurutan Berdasarkan Nama	Selection Sort	$O(n^2)$	$O(n^2)$
11.	Pengurutan Berdasarkan Kunjungan	Bubble Sort	$O(n^2)$	$O(n^2)$
12.	Statistik	Traversal	$O(n)$	$O(n)$
13.	Save CSV	Traversal	$O(n)$	$O(n)$

4.4 Kesimpulan Analisis Kompleksitas

Berdasarkan hasil analisis kompleksitas, sebagian besar fitur dalam Sistem Manajemen Pasien Klinik memiliki kompleksitas waktu linear $O(n)$ karena menggunakan proses traversal dan Linear Search pada struktur data ArrayList.

Fitur pengurutan menggunakan Bubble Sort dan Selection Sort memiliki kompleksitas $O(n^2)$ karena memanfaatkan nested loop dalam proses pengurutan data. Sementara itu, fitur pencarian berdasarkan ID memanfaatkan kombinasi Bubble Sort dan Binary Search untuk meningkatkan efisiensi pencarian pada data yang telah terurut.

Secara keseluruhan, algoritma yang digunakan sudah sesuai untuk skala data kecil hingga menengah dan mendukung kebutuhan pengelolaan data pasien secara sederhana dan efisien.

BAB V

PENGUJIAN

5.1 Pengujian Menu Ke-1 (CRUD)

Skenario menambahkan data pasien baru seperti ID, nama, kategori, status, dan jumlah kunjungan:

```
Pilih: 1
ID: 31
Nama: Mark
Kategori [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: Umum
Status [1 = Aktif, 0 = Nonaktif]: 1
Jumlah kunjungan: 1
Data berhasil disimpan ke CSV!
Data pasien berhasil ditambahkan!
```

Data pasien baru tersimpan pada CSV:

ID: 1	Nama: Alya	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 2	Nama: Bima	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 3	Nama: Citra	Kategori: Gigi	Status: Aktif	Kunjungan: 1
ID: 4	Nama: Doni	Kategori: THT	Status: Nonaktif	Kunjungan: 3
ID: 5	Nama: Eka	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 6	Nama: Ali	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 7	Nama: Aldo	Kategori: Anak	Status: Aktif	Kunjungan: 4
ID: 8	Nama: Ryul	Kategori: Mata	Status: Aktif	Kunjungan: 5
ID: 9	Nama: Yujin	Kategori: KIA	Status: Nonaktif	Kunjungan: 7
ID: 10	Nama: Dian	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 11	Nama: Daniella	Kategori: Gigi	Status: Aktif	Kunjungan: 6
ID: 12	Nama: Andrew	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 13	Nama: Clein	Kategori: Mata	Status: Nonaktif	Kunjungan: 3
ID: 14	Nama: Dominic	Kategori: Umum	Status: Aktif	Kunjungan: 5
ID: 15	Nama: Kyan	Kategori: Gigi	Status: Aktif	Kunjungan: 4
ID: 16	Nama: Magdalein	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 17	Nama: Silvy	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 18	Nama: Hana	Kategori: Umum	Status: Aktif	Kunjungan: 8
ID: 19	Nama: Grace	Kategori: KIA	Status: Nonaktif	Kunjungan: 3
ID: 20	Nama: Crish	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 21	Nama: Ilyas	Kategori: Mata	Status: Aktif	Kunjungan: 1
ID: 22	Nama: Asteer	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 23	Nama: Penelope	Kategori: THT	Status: Aktif	Kunjungan: 2
ID: 24	Nama: Zenya	Kategori: Umum	Status: Nonaktif	Kunjungan: 1
ID: 25	Nama: Upin	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 26	Nama: Itachi	Kategori: THT	Status: Aktif	Kunjungan: 8
ID: 27	Nama: Sanemi	Kategori: Mata	Status: Aktif	Kunjungan: 3
ID: 28	Nama: Kiryu	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 29	Nama: Hanabi	Kategori: KIA	Status: Nonaktif	Kunjungan: 9
ID: 30	Nama: Yuxiu	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 31	Nama: Mark	Kategori: Umum	Status: Aktif	Kunjungan: 1

Skenario apabila memasukkan ID yang sudah terdaftar:

Pilih: 1
ID: 14
ID: 14 sudah terdaftar! Gagal menambahkan pasien.

Skenario apabila memasukkan nama yang sudah terdaftar:

Pilih: 1
ID: 32
Nama: Mark
Nama: Mark sudah terdaftar! Gagal menambahkan pasien.

5.2 Pengujian Menu Ke-2 (CRUD)

Skenario menampilkan seluruh data pasien yang tersimpan di CSV:

```
Pilih: 2
ID: 1 | Nama: Alya | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 2 | Nama: Bima | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 3 | Nama: Citra | Kategori: Gigi | Status: Aktif | Kunjungan: 1
ID: 4 | Nama: Doni | Kategori: THT | Status: Nonaktif | Kunjungan: 3
ID: 5 | Nama: Eka | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 6 | Nama: Ali | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 7 | Nama: Aldo | Kategori: Anak | Status: Aktif | Kunjungan: 4
ID: 8 | Nama: Ryul | Kategori: Mata | Status: Aktif | Kunjungan: 5
ID: 9 | Nama: Yujin | Kategori: KIA | Status: Nonaktif | Kunjungan: 7
ID: 10 | Nama: Dian | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 11 | Nama: Daniella | Kategori: Gigi | Status: Aktif | Kunjungan: 6
ID: 12 | Nama: Andrew | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 13 | Nama: Clein | Kategori: Mata | Status: Nonaktif | Kunjungan: 3
ID: 14 | Nama: Dominic | Kategori: Umum | Status: Aktif | Kunjungan: 5
ID: 15 | Nama: Kyan | Kategori: Gigi | Status: Aktif | Kunjungan: 4
ID: 16 | Nama: Magdalein | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 17 | Nama: Silvy | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 18 | Nama: Hana | Kategori: Umum | Status: Aktif | Kunjungan: 8
ID: 19 | Nama: Grace | Kategori: KIA | Status: Nonaktif | Kunjungan: 3
ID: 20 | Nama: Crish | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 21 | Nama: Ilyas | Kategori: Mata | Status: Aktif | Kunjungan: 1
ID: 22 | Nama: Asteer | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 23 | Nama: Penelope | Kategori: THT | Status: Aktif | Kunjungan: 2
ID: 24 | Nama: Zenya | Kategori: Umum | Status: Nonaktif | Kunjungan: 1
ID: 25 | Nama: Upin | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 26 | Nama: Itachi | Kategori: THT | Status: Aktif | Kunjungan: 8
ID: 27 | Nama: Sanemi | Kategori: Mata | Status: Aktif | Kunjungan: 3
ID: 28 | Nama: Kiryu | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 29 | Nama: Hanabi | Kategori: KIA | Status: Nonaktif | Kunjungan: 9
ID: 30 | Nama: Yuxiu | Kategori: Umum | Status: Aktif | Kunjungan: 4
```

5.3 Pengujian Menu Ke-3 (CRUD)

Skenario mengedit kategori dan jumlah kunjungan pada pasien dengan ID 15:

```
Pilih: 3
Masukkan ID: 15
Nama: Kyan
Kategori baru [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: Mata
Status baru [1 = Aktif, 0 = Nonaktif]: 1
Jumlah kunjungan baru: 5
```

Tampilan data yang tersimpan setelah pengeditan data pasien dengan ID 15:

```
Pilih: 2
ID: 1 | Nama: Alya | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 2 | Nama: Bima | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 3 | Nama: Citra | Kategori: Gigi | Status: Aktif | Kunjungan: 1
ID: 4 | Nama: Doni | Kategori: THT | Status: Nonaktif | Kunjungan: 3
ID: 5 | Nama: Eka | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 6 | Nama: Ali | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 7 | Nama: Aldo | Kategori: Anak | Status: Aktif | Kunjungan: 4
ID: 8 | Nama: Ryul | Kategori: Mata | Status: Aktif | Kunjungan: 5
ID: 9 | Nama: Yujin | Kategori: KIA | Status: Nonaktif | Kunjungan: 7
ID: 10 | Nama: Dian | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 11 | Nama: Daniella | Kategori: Gigi | Status: Aktif | Kunjungan: 6
ID: 12 | Nama: Andrew | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 13 | Nama: Clein | Kategori: Mata | Status: Nonaktif | Kunjungan: 3
ID: 14 | Nama: Dominic | Kategori: Umum | Status: Aktif | Kunjungan: 5
ID: 15 | Nama: Kyan | Kategori: Mata | Status: Nonaktif | Kunjungan: 5
ID: 16 | Nama: Magdalein | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 17 | Nama: Silvy | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 18 | Nama: Hana | Kategori: Umum | Status: Aktif | Kunjungan: 8
ID: 19 | Nama: Grace | Kategori: KIA | Status: Nonaktif | Kunjungan: 3
ID: 20 | Nama: Crish | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 21 | Nama: Ilyas | Kategori: Mata | Status: Aktif | Kunjungan: 1
ID: 22 | Nama: Asteer | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 23 | Nama: Penelope | Kategori: THT | Status: Aktif | Kunjungan: 2
ID: 24 | Nama: Zenya | Kategori: Umum | Status: Nonaktif | Kunjungan: 1
ID: 25 | Nama: Upin | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 26 | Nama: Itachi | Kategori: THT | Status: Aktif | Kunjungan: 8
ID: 27 | Nama: Sanemi | Kategori: Mata | Status: Aktif | Kunjungan: 3
ID: 28 | Nama: Kiryu | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 29 | Nama: Hanabi | Kategori: KIA | Status: Nonaktif | Kunjungan: 9
ID: 30 | Nama: Yuxiu | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 31 | Nama: Mark | Kategori: Umum | Status: Aktif | Kunjungan: 1
```

5.4 Pengujian Menu Ke-4 (CRUD)

Skenario menonaktifkan status pasien dengan ID 31:

Pilih: 4
ID: 31
Data berhasil disimpan ke CSV!

Tampilan output data pasien dengan ID 31 setelah dinonaktifkan:

Pilih: 2

ID: 1	Nama: Alya	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 2	Nama: Bima	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 3	Nama: Citra	Kategori: Gigi	Status: Aktif	Kunjungan: 1
ID: 4	Nama: Doni	Kategori: THT	Status: Nonaktif	Kunjungan: 3
ID: 5	Nama: Eka	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 6	Nama: Ali	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 7	Nama: Aldo	Kategori: Anak	Status: Aktif	Kunjungan: 4
ID: 8	Nama: Ryul	Kategori: Mata	Status: Aktif	Kunjungan: 5
ID: 9	Nama: Yujin	Kategori: KIA	Status: Nonaktif	Kunjungan: 7
ID: 10	Nama: Dian	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 11	Nama: Daniella	Kategori: Gigi	Status: Aktif	Kunjungan: 6
ID: 12	Nama: Andrew	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 13	Nama: Clein	Kategori: Mata	Status: Nonaktif	Kunjungan: 3
ID: 14	Nama: Dominic	Kategori: Umum	Status: Aktif	Kunjungan: 5
ID: 15	Nama: Kyan	Kategori: Mata	Status: Nonaktif	Kunjungan: 5
ID: 16	Nama: Magdalein	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 17	Nama: Silvy	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 18	Nama: Hana	Kategori: Umum	Status: Aktif	Kunjungan: 8
ID: 19	Nama: Grace	Kategori: KIA	Status: Nonaktif	Kunjungan: 3
ID: 20	Nama: Crish	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 21	Nama: Ilyas	Kategori: Mata	Status: Aktif	Kunjungan: 1
ID: 22	Nama: Asteer	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 23	Nama: Penelope	Kategori: THT	Status: Aktif	Kunjungan: 2
ID: 24	Nama: Zenya	Kategori: Umum	Status: Nonaktif	Kunjungan: 1
ID: 25	Nama: Upin	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 26	Nama: Itachi	Kategori: THT	Status: Aktif	Kunjungan: 8
ID: 27	Nama: Sanemi	Kategori: Mata	Status: Aktif	Kunjungan: 3
ID: 28	Nama: Kiryu	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 29	Nama: Hanabi	Kategori: KIA	Status: Nonaktif	Kunjungan: 9
ID: 30	Nama: Yuxiu	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 31	Nama: Mark	Kategori: Umum	Status: Nonaktif	Kunjungan: 1

5.5 Pengujian Menu Ke-5 (Searching)

Skenario mencari data pasien dengan nama:

Pilih: 5
Nama: Grace
ID: 19 | Nama: Grace | Kategori: KIA | Status: Nonaktif | Kunjungan: 3

Skenario mencari data pasien dengan nama apabila nama tidak tersimpan di CSV:

Pilih: 5
Nama: Haidar
Nama Pasien Haidar Tidak Ditemukan.

5.6 Pengujian Menu Ke-6 (Searching)

Skenario mencari data pasien dengan ID:

Pilih: 6
ID: 12
ID: 12 | Nama: Andrew | Kategori: Anak | Status: Nonaktif | Kunjungan: 1

Skenario mencari data pasien dengan ID apabila ID tidak tersimpan di CSV:

Pilih: 6
ID: 34
ID Pasien 34 Tidak ditemukan

5.7 Pengujian Menu Ke-7 (Searching)

Skenario mencari data pasien dengan jenis kategori:

Pilih: 7
Masukkan Kategori yang Dicari [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: Anak
ID: 2 | Nama: Bima | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 7 | Nama: Aldo | Kategori: Anak | Status: Aktif | Kunjungan: 4
ID: 12 | Nama: Andrew | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 17 | Nama: Silvy | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 22 | Nama: Asteer | Kategori: Anak | Status: Nonaktif | Kunjungan: 1

Skenario mencari data pasien dengan jenis kategori apabila kategori tidak tersimpan di CSV:

Pilih: 7
Masukkan Kategori yang Dicari [Umum, Anak, Spesialis, KIA, THT, Gigi, Mata]: Spesialis
Pasien dengan Kategori Spesialis Tidak Ditemukan.

5.8 Pengujian Menu Ke-8 (Searching)

Skenario mencari data pasien dengan jenis status aktif:

```
Pilih: 8
Masukkan Status yang Dicari [1 = Aktif, 0 = Nonaktif]: 1
ID: 1 | Nama: Alya | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 3 | Nama: Citra | Kategori: Gigi | Status: Aktif | Kunjungan: 1
ID: 5 | Nama: Eka | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 6 | Nama: Ali | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 7 | Nama: Aldo | Kategori: Anak | Status: Aktif | Kunjungan: 4
ID: 8 | Nama: Ryul | Kategori: Mata | Status: Aktif | Kunjungan: 5
ID: 10 | Nama: Dian | Kategori: Umum | Status: Aktif | Kunjungan: 4
ID: 11 | Nama: Daniella | Kategori: Gigi | Status: Aktif | Kunjungan: 6
ID: 14 | Nama: Dominic | Kategori: Umum | Status: Aktif | Kunjungan: 5
ID: 16 | Nama: Magdalein | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 18 | Nama: Hana | Kategori: Umum | Status: Aktif | Kunjungan: 8
ID: 20 | Nama: Crish | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 21 | Nama: Ilyas | Kategori: Mata | Status: Aktif | Kunjungan: 1
ID: 23 | Nama: Penelope | Kategori: THT | Status: Aktif | Kunjungan: 2
ID: 25 | Nama: Upin | Kategori: Mata | Status: Aktif | Kunjungan: 4
ID: 26 | Nama: Itachi | Kategori: THT | Status: Aktif | Kunjungan: 8
ID: 27 | Nama: Sanemi | Kategori: Mata | Status: Aktif | Kunjungan: 3
ID: 28 | Nama: Kiryu | Kategori: Umum | Status: Aktif | Kunjungan: 2
ID: 30 | Nama: Yuxiu | Kategori: Umum | Status: Aktif | Kunjungan: 4
```

Skenario mencari data pasien dengan jenis status nonaktif:

```
Pilih: 8
Masukkan Status yang Dicari [1 = Aktif, 0 = Nonaktif]: 0
ID: 2 | Nama: Bima | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 4 | Nama: Doni | Kategori: THT | Status: Nonaktif | Kunjungan: 3
ID: 9 | Nama: Yujin | Kategori: KIA | Status: Nonaktif | Kunjungan: 7
ID: 12 | Nama: Andrew | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 13 | Nama: Clein | Kategori: Mata | Status: Nonaktif | Kunjungan: 3
ID: 15 | Nama: Kyan | Kategori: Mata | Status: Nonaktif | Kunjungan: 5
ID: 17 | Nama: Silvy | Kategori: Anak | Status: Nonaktif | Kunjungan: 5
ID: 19 | Nama: Grace | Kategori: KIA | Status: Nonaktif | Kunjungan: 3
ID: 22 | Nama: Asteer | Kategori: Anak | Status: Nonaktif | Kunjungan: 1
ID: 24 | Nama: Zenya | Kategori: Umum | Status: Nonaktif | Kunjungan: 1
ID: 29 | Nama: Hanabi | Kategori: KIA | Status: Nonaktif | Kunjungan: 9
ID: 31 | Nama: Mark | Kategori: Umum | Status: Nonaktif | Kunjungan: 1
```

5.9 Pengujian Menu Ke-9 (Sorting)

Skenario mengurutkan data pasien berdasarkan ID:

Pilih: 9

=== Hasil Sort Berdasarkan ID ===

ID: 1	Nama: Alya	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 2	Nama: Bima	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 3	Nama: Citra	Kategori: Gigi	Status: Aktif	Kunjungan: 1
ID: 4	Nama: Doni	Kategori: THT	Status: Nonaktif	Kunjungan: 3
ID: 5	Nama: Eka	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 6	Nama: Ali	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 7	Nama: Aldo	Kategori: Anak	Status: Aktif	Kunjungan: 4
ID: 8	Nama: Ryul	Kategori: Mata	Status: Aktif	Kunjungan: 5
ID: 9	Nama: Yujin	Kategori: KIA	Status: Nonaktif	Kunjungan: 7
ID: 10	Nama: Dian	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 11	Nama: Daniella	Kategori: Gigi	Status: Aktif	Kunjungan: 6
ID: 12	Nama: Andrew	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 13	Nama: Clein	Kategori: Mata	Status: Nonaktif	Kunjungan: 3
ID: 14	Nama: Dominic	Kategori: Umum	Status: Aktif	Kunjungan: 5
ID: 15	Nama: Kyan	Kategori: Mata	Status: Nonaktif	Kunjungan: 5
ID: 16	Nama: Magdalein	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 17	Nama: Silvy	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 18	Nama: Hana	Kategori: Umum	Status: Aktif	Kunjungan: 8
ID: 19	Nama: Grace	Kategori: KIA	Status: Nonaktif	Kunjungan: 3
ID: 20	Nama: Crish	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 21	Nama: Ilyas	Kategori: Mata	Status: Aktif	Kunjungan: 1
ID: 22	Nama: Asteer	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 23	Nama: Penelope	Kategori: THT	Status: Aktif	Kunjungan: 2
ID: 24	Nama: Zenya	Kategori: Umum	Status: Nonaktif	Kunjungan: 1
ID: 25	Nama: Upin	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 26	Nama: Itachi	Kategori: THT	Status: Aktif	Kunjungan: 8
ID: 27	Nama: Sanemi	Kategori: Mata	Status: Aktif	Kunjungan: 3
ID: 28	Nama: Kiryu	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 29	Nama: Hanabi	Kategori: KIA	Status: Nonaktif	Kunjungan: 9
ID: 30	Nama: Yuxiu	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 31	Nama: Mark	Kategori: Umum	Status: Nonaktif	Kunjungan: 1

5.10 Pengujian Menu Ke-10 (Sorting)

Skenario mengurutkan data pasien berdasarkan nama (Ascending):

Pilih: 10

=== Hasil Sort Berdasarkan Nama ===

ID: 7	Nama: Aldo	Kategori: Anak	Status: Aktif	Kunjungan: 4
ID: 6	Nama: Ali	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 1	Nama: Alya	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 12	Nama: Andrew	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 22	Nama: Asteer	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 2	Nama: Bima	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 3	Nama: Citra	Kategori: Gigi	Status: Aktif	Kunjungan: 1
ID: 13	Nama: Clein	Kategori: Mata	Status: Nonaktif	Kunjungan: 3
ID: 20	Nama: Crish	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 11	Nama: Daniella	Kategori: Gigi	Status: Aktif	Kunjungan: 6
ID: 10	Nama: Dian	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 14	Nama: Dominic	Kategori: Umum	Status: Aktif	Kunjungan: 5
ID: 4	Nama: Doni	Kategori: THT	Status: Nonaktif	Kunjungan: 3
ID: 5	Nama: Eka	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 19	Nama: Grace	Kategori: KIA	Status: Nonaktif	Kunjungan: 3
ID: 18	Nama: Hana	Kategori: Umum	Status: Aktif	Kunjungan: 8
ID: 29	Nama: Hanabi	Kategori: KIA	Status: Nonaktif	Kunjungan: 9
ID: 21	Nama: Ilyas	Kategori: Mata	Status: Aktif	Kunjungan: 1
ID: 26	Nama: Itachi	Kategori: THT	Status: Aktif	Kunjungan: 8
ID: 28	Nama: Kiryu	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 15	Nama: Kyan	Kategori: Mata	Status: Nonaktif	Kunjungan: 5
ID: 16	Nama: Magdalein	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 31	Nama: Mark	Kategori: Umum	Status: Nonaktif	Kunjungan: 1
ID: 23	Nama: Penelope	Kategori: THT	Status: Aktif	Kunjungan: 2
ID: 8	Nama: Ryul	Kategori: Mata	Status: Aktif	Kunjungan: 5
ID: 27	Nama: Sanemi	Kategori: Mata	Status: Aktif	Kunjungan: 3
ID: 17	Nama: Silvy	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 25	Nama: Upin	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 9	Nama: Yujin	Kategori: KIA	Status: Nonaktif	Kunjungan: 7
ID: 30	Nama: Yuxiu	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 24	Nama: Zenya	Kategori: Umum	Status: Nonaktif	Kunjungan: 1

5.11 Pengujian Menu Ke-11 (Sorting)

Skenario mengurutkan data pasien berdasarkan jumlah kunjungan (Descending):

Pilih: 11

=== Hasil Sort Berdasarkan Kunjungan Terbanyak ===

ID: 29	Nama: Hanabi	Kategori: KIA	Status: Nonaktif	Kunjungan: 9
ID: 18	Nama: Hana	Kategori: Umum	Status: Aktif	Kunjungan: 8
ID: 26	Nama: Itachi	Kategori: THT	Status: Aktif	Kunjungan: 8
ID: 9	Nama: Yujin	Kategori: KIA	Status: Nonaktif	Kunjungan: 7
ID: 11	Nama: Daniella	Kategori: Gigi	Status: Aktif	Kunjungan: 6
ID: 2	Nama: Bima	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 14	Nama: Dominic	Kategori: Umum	Status: Aktif	Kunjungan: 5
ID: 15	Nama: Kyan	Kategori: Mata	Status: Nonaktif	Kunjungan: 5
ID: 8	Nama: Ryul	Kategori: Mata	Status: Aktif	Kunjungan: 5
ID: 17	Nama: Silvy	Kategori: Anak	Status: Nonaktif	Kunjungan: 5
ID: 7	Nama: Aldo	Kategori: Anak	Status: Aktif	Kunjungan: 4
ID: 20	Nama: Crish	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 10	Nama: Dian	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 5	Nama: Eka	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 25	Nama: Upin	Kategori: Mata	Status: Aktif	Kunjungan: 4
ID: 30	Nama: Yuxiu	Kategori: Umum	Status: Aktif	Kunjungan: 4
ID: 13	Nama: Clein	Kategori: Mata	Status: Nonaktif	Kunjungan: 3
ID: 4	Nama: Doni	Kategori: THT	Status: Nonaktif	Kunjungan: 3
ID: 19	Nama: Grace	Kategori: KIA	Status: Nonaktif	Kunjungan: 3
ID: 27	Nama: Sanemi	Kategori: Mata	Status: Aktif	Kunjungan: 3
ID: 6	Nama: Ali	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 1	Nama: Alya	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 28	Nama: Kiryu	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 16	Nama: Magdalein	Kategori: Umum	Status: Aktif	Kunjungan: 2
ID: 23	Nama: Penelope	Kategori: THT	Status: Aktif	Kunjungan: 2
ID: 12	Nama: Andrew	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 22	Nama: Asteer	Kategori: Anak	Status: Nonaktif	Kunjungan: 1
ID: 3	Nama: Citra	Kategori: Gigi	Status: Aktif	Kunjungan: 1
ID: 21	Nama: Ilyas	Kategori: Mata	Status: Aktif	Kunjungan: 1
ID: 31	Nama: Mark	Kategori: Umum	Status: Nonaktif	Kunjungan: 1
ID: 24	Nama: Zenya	Kategori: Umum	Status: Nonaktif	Kunjungan: 1

5.12 Pengujian Menu Ke-12 (Sorting)

Skenario menampilkan statistik data pasien yang tersimpan di CSV:

```
01. Menu
Pilih: 12
Total: 31
Aktif: 19
```

BAB VI

KESIMPULAN

6.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan analisis yang telah dilakukan pada Sistem Manajemen Pasien Klinik, dapat ditarik beberapa kesimpulan sebagai berikut:

- 1) Sistem berhasil dikembangkan dengan fitur pengelolaan data pasien secara lengkap melalui operasi CRUD (*Create, Read, Update, Delete*), di mana fitur *soft delete* diterapkan untuk menjaga integritas riwayat data pasien tanpa menghapus data secara permanen
- 2) Struktur data ArrayList terbukti menjadi pilihan yang tepat sebagai wadah penyimpanan data pasien selama program berjalan, karena sifatnya yang dinamis memudahkan penambahan data tanpa batasan kapasitas tetap, sekaligus mendukung operasi traversal dan modifikasi data secara efisien.
- 3) Persistensi data berbasis file CSV berhasil diimplementasikan menggunakan mekanisme *immediate persistence*, sehingga setiap perubahan data langsung tersimpan dan tidak hilang saat aplikasi ditutup.
- 4) Algoritma pencarian yang diterapkan meliputi *Linear Search* (yang efektif pada data tidak terurut) untuk pencarian berdasarkan nama, kategori, dan status serta *Binary Search* untuk pencarian berdasarkan ID yang bekerja lebih efisien dengan kompleksitas $O(\log n)$ pada data yang telah diurutkan terlebih dahulu.
- 5) Algoritma pengurutan yang digunakan meliputi *Bubble Sort* untuk mengurutkan data berdasarkan ID (*ascending*) dan jumlah kunjungan (*descending*), serta *Selection Sort* untuk mengurutkan

nama pasien secara alfabetis. Keduanya memiliki kompleksitas $O(n^2)$, namun tetap memadai untuk skala data kecil hingga menengah yang umum ditemukan di lingkungan klinik.

- 6) Analisis kompleksitas menunjukkan bahwa sebagian besar fitur sistem berjalan pada tingkat efisiensi $O(n)$, sementara fitur pengurutan berada pada $O(n^2)$. Secara keseluruhan, algoritma yang dipilih sudah sesuai dan cukup optimal untuk kebutuhan pengelolaan data pasien dalam skala klinik.
- 7) Fitur statistik yang disediakan mampu memberikan informasi ringkas mengenai total pasien dan jumlah pasien aktif, sehingga dapat mendukung pengambilan keputusan operasional klinik secara lebih cepat dan akurat.

Dengan demikian, Sistem Manajemen Pasien Klinik yang dikembangkan mampu menjawab permasalahan pengelolaan data yang selama ini dilakukan secara manual, sekaligus menjadi implementasi nyata dari konsep algoritma dan struktur data dalam penyelesaian masalah di dunia nyata.